INTERVIEW READINESS PROGRAM • SESSION 3

# More Problems

# What we have covered so far

SESSION 1

## Judging an approach

- Why DSA still matters when AI writes code
- Big-O and the growth ladder
- Space counts — including the recursion stack

SESSION 2

## Reading the clues

- The six-step problem-solving loop
- Constraints choose the pattern
- Prefix sum + hash map
- Monotonic stack
- Merge sort as a counting tool

BEFORE WE START

# Homework from last session

HW 1

## Shortest subarray with sum ≥ K

Shortest contiguous run whose sum reaches K. Values may be negative. −1 if none.

```
a = [2,-1,2,-1,2,3], K = 5  →  2
a = [1,-1,1], K = 4  →  -1
```

HW 2

## Smaller values to the right

For every position, how many strictly smaller values sit after it.

```
a = [5,2,6,1,2]  →  [3,1,2,0,0]
a = [2,2]  →  [0,0]
```

# Shortest subarray with sum ≥ K

Given an integer array a and a positive K, return the length of the shortest non-empty contiguous subarray whose sum is at least K.

Values may be negative. Return −1 if no such subarray exists.

## Constraints

- 1 ≤ n ≤ 100,000; 1 ≤ K ≤ $10^9$
- Values are integers within ±$10^9$.
- Return a length, not the sum or the indices.

```
a = [2,-1,2,-1,2,3], K = 5
Output: 2

a = [1,-1,1], K = 4
Output: -1
```

## What the example means

In the first case the final [2,3] has sum 5 and length 2; no single value reaches 5. In the second, no contiguous subarray reaches 4.

# Smaller values to the right

Given an integer array a, return an array ans of the same length. For each position i, ans[i] counts the positions j > i where a[j] is strictly less than a[i].

Count each occurrence separately. Equal values do not count.

## Constraints

- $0 \le n \le 100{,}000$; values within $\pm 10^9$
- Return counts in the original input order.
- For an empty array, return an empty array.

```
a = [5,2,6,1,2]
Output: [3,1,2,0,0]

a = [2,2]
Output: [0,0]
```

## What the example means

For 5 the smaller values to its right are 2, 1, 2. For the first 2, only 1. For 6, both 1 and 2. The final 1 and 2 have nothing smaller after them. In [2,2], equality counts neither.

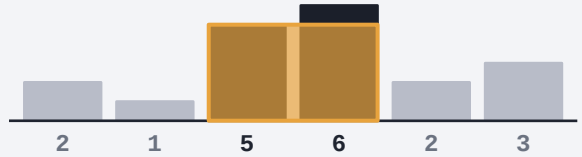
QUESTION 1

# Largest rectangle in a histogram

Given bar heights, each of width 1, return the area of the largest rectangle that fits entirely inside the histogram.

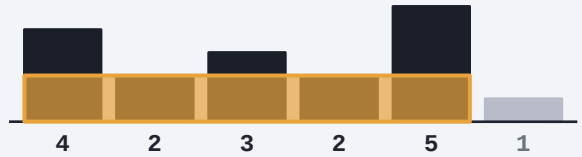
The rectangle must be axis-aligned and rest on the baseline.

`[2,1,5,6,2,3]` → **10**




*Tall and narrow: two neighbours of height ≥ 5.*

`[4,2,3,2,5,1]` → **10**




*Short and wide: height 2 across five bars, taller bars in between.*

## Constraints

- $1 \le n \le 100{,}000$; $0 \le \text{height} \le 10^9$
- Return the area, not the bars.
- A single bar is a valid rectangle.

DISCUSSION

# Largest rectangle: brute force vs. stack

## Brute force: $O(n^2)$

```
best ← 0
for i in 0..n−1:
  for j in i..n−1:
    area ← min(h[i], h[j]) × (j − i + 1)
    best ← max(area, best)
```

## Optimized: monotonic stack, O(n)

```
stack holds (height, start)
for each bar i (then a final 0):
  start ← i
  while top.height > h[i]:
    pop (hgt, s); best ← max(best, hgt × (i − s))
    start ← s
  push (h[i], start)
```

# Stack simulation

| | | |
|---|---|---|
| i=0 | 2 arrives, nothing to compare. Push (2, 0). | [(2,0)] |
| i=1 | 1 arrives. (2,0) is taller: pop, area 2 × (1 − 0) = 2. The new bar inherits start 0. Push (1, 0). | [(1,0)] |
| i=2..3 | 5 and 6 are rising. Nothing is finished. Push (5, 2), then (6, 3). | [(1,0) (5,2) (6,3)] |
| i=4 | 2 arrives. Pop (6,3): 6 × (4 − 3) = 6. Pop (5,2): 5 × (4 − 2) = 10. Inherit start 2. Push (2, 2). | [(1,0) (2,2)] |
| i=5 | 3 rises. Push (3, 5). | [(1,0) (2,2) (3,5)] |
| end | Empty the stack (3,5): 3 × 1 = 3. (2,2): 2 × 4 = 8. (1,0): 1 × 6 = 6. | best = 10 |

# Pop when a smaller bar arrives

```
stack ← empty   // (height, start), heights rising
best  ← 0

for i from 0 to n:  // i = n: sentinel, height 0
    h     ← (i = n) ? 0 : height[i]
    start ← i
    while stack not empty and top.height > h:
        (ht, st) ← pop
        best  ← max(best, ht × (i − st))
        start ← st     // h reaches back to st
    push (h, start)

return best
```

**Time O(n). Extra space O(n).**

Each bar is pushed once and popped once. A popped tuple already knows its left edge; the bar that popped it is its right edge.

QUESTION 2

# Decode a nested string

An encoded string uses the rule k[s], meaning the string s is repeated exactly k times. Encodings can be nested.

Return the decoded string.

```
s = "3[a2[c]]"
Output: "accaccacc"

s = "2[abc]3[cd]ef"
Output: "abcabccdcdcdef"
```

### Constraints

- 1 ≤ length ≤ 30; k is a positive integer, possibly multi-digit.
- Input is always valid: brackets are balanced.
- Letters are lowercase; no digits appear outside a count.
- Output length is at most $10^5$.

### What the example means

In the first case the inner 2[c] becomes cc, so the outer bracket holds acc, repeated 3 times. In the second, two groups are decoded in sequence and ef is appended as-is.

# DRY RUN · 3[A2[C]]

| read | what we do | saved (count, text before) | current text |
|---|---|---|---|
| 3 | A count. Remember it: k = 3. | — | "" |
| [ | A group opens. Save (3, "") and start a fresh current text. | (3, "") | "" |
| a | A letter inside a group. Append to the current text. | (3, "") | "a" |
| 2 | Another count: k = 2. | (3, "") | "a" |
| [ | Nested group opens. Save (2, "a") and start fresh. | (3, "") (2, "a") | "" |
| c | Append. | (3, "") (2, "a") | "c" |
| ] | Inner group closes. Take back (2, "a"): "a" + "c" × 2. | (3, "") | "acc" |
| ] | Outer group closes. Take back (3, ""): "" + "acc" × 3. | — | "accaccacc" |

```
saved ← empty stack   // (count, text before '[')
cur   ← ""            // text inside the open group
k     ← 0

for each character c:
    if c is a digit:  k ← k × 10 + digit(c)
    else if c = '[':  push (k, cur); k ← 0; cur ← ""
    else if c = ']':  (n, before) ← pop
                      cur ← before + (cur × n)
    else:             cur ← cur + c

return cur
```

**Time O(length of output).**

Not O(n) on the input: 10[a] reads 5 characters and writes 10. Say that out loud. Space is the depth of nesting plus the output.

**Try these before running code**

```
"3[a]2[bc]" → aaabcbc
"10[a]" → ten a's
"abc" → abc
"2[2[2[x]]]" → xxxxxxxx
```

# Two to take home before we move on

## Q1 Evaluate an expression

A string with non-negative integers, + − × ÷, parentheses and spaces. Return its value. Usual precedence; integer division truncates toward zero.

```
"3 + 5 * (2 - 8)"  →  -27
"10 / 2 - 3"       →  2
"(1+(4+5+2)-3)+(6+8)"  →  23
```

- $1 \le \text{length} \le 10^5$; the result fits in 32 bits.
- The input is always a valid expression.

*Two kinds of unfinished work: numbers waiting for an operator, and operators waiting until precedence lets them run. Where does each kind wait?*

## Q2 Trapping rain water

Bars of width 1 with the given heights. After it rains, how many units of water are held between them?

```
[0,1,0,2,1,0,1,3,2,1,2,1]  →  6
[4,2,0,3,2,5]  →  9
```

- $1 \le n \le 20{,}000$; $0 \le \text{height} \le 10^5$.
- The first and last bars never hold water.

*Which bar closes a pool, and how far back does that pool reach? You have solved a close relative of this today.*

READING THE CLUES

# How to tell a problem can be solved using a stack

1

### Each element waits for something later

Its answer depends on the next bigger / smaller / warmer thing to arrive.

`next greater element · histogram`
`stock span · daily temperatures`

2

### Things open and close, most recent first

Brackets, tags, k[...] groups, nested calls. Whatever opened last closes first.

`valid parentheses · decode string`
`expression evaluation`

3

### You compare against the latest survivor

Earlier items get eliminated; each new arrival is tested against the most recent one left.

`asteroid collision · remove K digits`
`remove adjacent duplicates`

4

### You need to undo the last step

Backspace, '..' in a path, reverting a move. Only the newest action can be undone.

`backspace string compare`
`simplify path · undo history`

5

### Recursion you want to run without recursion

DFS, tree traversals, anything with a natural call stack you would rather control yourself.

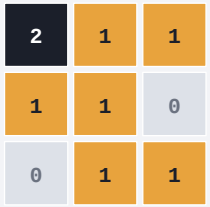
`iterative inorder · iterative DFS`
`flatten nested list`

QUESTION 3

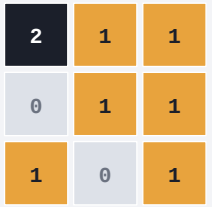
# Rotting oranges

A grid holds empty cells (0), fresh oranges (1) and rotten oranges (2). Every minute, each rotten orange rots the fresh oranges directly above, below, left and right of it.

Return the number of minutes until no fresh orange remains, or −1 if that never happens.




*Output: 4*




*Output: -1*




## Constraints

- 1 ≤ rows, cols ≤ 10; cells are 0, 1 or 2
- Rotting spreads to all four neighbours at once.
- If there are no fresh oranges to begin with, answer 0.

## What the examples mean

Left: rot spreads one ring per minute and reaches the far corner in 4.
Right: the orange at the bottom-left is walled off by empty cells, so it never rots.

# Rot spreads in waves



**minute 0**

one rotten orange at (0,0)

**minute 1**

(0,0) rots (0,1), (1,0)

**minute 2**

those two rot (0,2), (1,1)

**minute 3**

(1,1) rots (2,1)

**minute 4**

(2,1) rots (2,2). Done.

# All sources at once, one wave per minute

```
q ← empty queue;  fresh ← 0
for each cell (r, c):
    if grid[r][c] = 2: push (r, c)        // all sources
    if grid[r][c] = 1: fresh ← fresh + 1

minutes ← 0
while q not empty and fresh > 0:
    repeat size(q) times:                  // one wave
        (r, c) ← pop
        for each 4-neighbour (nr, nc) inside the grid:
            if grid[nr][nc] = 1:
                grid[nr][nc] ← 2;  fresh ← fresh − 1
                push (nr, nc)
    minutes ← minutes + 1

return (fresh = 0) ? minutes : −1
```

**Time O(rows × cols). Space the same.**

Every cell enters the queue at most once and is popped once. The 'repeat size(q) times' loop is what turns a plain queue into a minute counter.

**Try these before running code**

```
[[0,2]]  →  0
[[1]]  →  -1
[[2,1,1],[0,1,1],[1,0,1]]  →  -1
[[2,2],[1,1]]  →  1
```

*This is breadth-first search with many starting points at once. Graphs week will give it the name; today it is 'process in arrival order, one wave at a time.' Mutating the grid in place is the visited set.*

QUESTION 4

# Minimum meeting rooms

You are given meetings as [start, end) intervals. Return the minimum number of rooms needed so that no two meetings that overlap in time share a room.

A meeting that ends at time t and one that starts at time t may use the same room.

### Constraints

- $1 \le n \le 10^4$; $0 \le \text{start} < \text{end} \le 10^6$
- Meetings are not given in any order.
- Return a single integer: the number of rooms.

```
[[0,30],[5,10],[15,20]]
Output: 2

[[1,5],[2,6],[3,7],[4,8]]
Output: 4
```

### What the example means

First case: [5,10] and [15,20] can share a room, [0,30] needs its own. Second: all four meetings overlap at time 4, so four rooms.

# Reuse the room that frees up first

```
sort meetings by start
ends  ← empty min-heap   // end times in use
rooms ← 0

for each (start, end) in sorted order:
    if ends not empty and min(ends) ≤ start:
        pop min          // that room is free
    push end
    rooms ← max(rooms, size(ends))

return rooms
```

**Dry run · [[0,30],[5,10],[15,20]]**

| meeting | decision | ends | rooms |
|---|---|---|---|
| [0,30] | heap empty → new room | {30} | 1 |
| [5,10] | min 30 > 5 → new room | {10, 30} | 2 |
| [15,20] | min 10 ≤ 15 → reuse | {20, 30} | 2 |

*The room most likely to be free is the one whose meeting ends earliest — so we only ever need the smallest end time.*

**Time O(n log n). Space O(n).**

Sorting is n log n; each meeting does at most one pop and one push on the heap, log n each. The shape to recognise: 'repeatedly take the minimum, remove it, insert something new' — that is a priority queue. Without a heap: sort starts and ends separately and sweep with two pointers, also O(n log n).